

3719. Longest Balanced Subarray I

Solved 

Medium

 Topics

 Companies

 Hint

You are given an integer array `nums`.

A **subarray** is called **balanced** if the number of **distinct even** numbers in the subarray is equal to the number of **distinct odd** numbers.

Return the length of the **longest** balanced subarray.

Example 1:

Input: `nums = [2, 5, 4, 3]`

Output: 4

Explanation:

- The longest balanced subarray is `[2, 5, 4, 3]`.
- It has 2 distinct even numbers `[2, 4]` and 2 distinct odd numbers `[5, 3]`. Thus, the answer is 4.

Example 2:

Input: `nums = [3,2,2,5,4]`

Output: 5

Explanation:

- The longest balanced subarray is `[3, 2, 2, 5, 4]`.
- It has 2 distinct even numbers `[2, 4]` and 2 distinct odd numbers `[3, 5]`. Thus, the answer is 5.

Example 3:

Input: `nums = [1,2,3,2]`

Output: 3

Explanation:

- The longest balanced subarray is `[2, 3, 2]`.
- It has 1 distinct even number `[2]` and 1 distinct odd number `[3]`. Thus, the answer is 3.

Constraints:

- `1 <= nums.length <= 1500`
- `1 <= nums[i] <= 105`

Python:

class Solution:

def longestBalanced(self, nums: List[int]) -> int:

n = len(nums)

res = 0

seen = [0] * (max(nums) + 1)

for i in range(n):

A = [0, 0]

```

    for j in range(i, n):
        val = nums[j]
        if seen[val] != i + 1:
            seen[val] = i + 1
            A[val & 1] += 1

        if A[0] == A[1]:
            res = max(res, j - i + 1)

```

```

return res

```

JavaScript:

```

const longestBalanced = nums => {
    const n = nums.length;
    let res = 0;

    const seen = new Int32Array(Math.max(...nums) + 1);

    for (let i = 0; i < n; i++) {
        const A = [0, 0];

        for (let j = i; j < n; j++) {
            const val = nums[j];
            if (seen[val] !== i + 1) {
                seen[val] = i + 1;
                A[val & 1]++;
            }

            if (A[0] === A[1])
                res = Math.max(res, j - i + 1);
        }
    }

    return res;
};

```

Java:

```

class Solution {
    private static int[] seen = new int[100001];
    private static int leet = 0;
    public int longestBalanced(int[] nums) {
        leet++;
        int n = nums.length;
        int res = 0;

```

```
for (int i = 0; i < n; i++) {  
    int[] A = new int[2];  
    int marker = (leet << 16) | (i + 1);  
    for (int j = i; j < n; j++) {  
        int val = nums[j];  
        if (seen[val] != marker) {  
            seen[val] = marker;  
            A[val & 1]++;  
        }  
  
        if (A[0] == A[1])  
            res = Math.max(res, j - i + 1);  
    }  
}  
  
return res;  
}
```